

MYNTRA-STYLE STORE WEBSITE DOCUMENTATION

PURPOSE

This document explains the current Myntra-style storefront in 'module4/myntra-style-store' so that a new learner, reviewer, or mentor can quickly understand:

- what the website does,
- how the frontend and backend are connected,
- how to run it locally,
- what is already implemented,
- and which improvements are suggested in 'module4/top-up-ideas'.

PROJECT SNAPSHOT

The project is a small full-stack demo store inspired by Myntra's merchandising style. It uses:

- **Frontend:** React 19 with Vite
- **Backend:** Express 5 on Node.js
- **Communication:** REST API over 'http://localhost:5174/api/products'
- **UI style:** marketing-led landing page with product cards, category blocks, hero section, and architecture summary panels

The application is intentionally lightweight and well suited for learning component structure, API consumption, responsive layout, and simple documentation practices.

FOLDER STRUCTURE

```
module4/  
  myntra-style-store/  
    backend/  
      index.js  
      package.json  
    frontend/  
      package.json  
      src/  
        main.jsx  
        App.jsx  
        App.css  
        index.css  
  top-up-ideas/  
    modification-ideas.md  
  docs/  
    codebase-understanding.md  
    report-prompts.md  
    summary-prompts.md
```

WHAT THE WEBSITE CURRENTLY DOES

1. Frontend experience

The frontend renders a single-page shopping showcase with these major sections:

- **Sticky header** with navigation links and a login/explore CTA
- **Hero section** describing the concept and showing runtime status
- **Live categories** generated from the fetched product data
- **Product spotlight grid** showing curated items
- **System map & telemetry** cards summarizing the stack
- **Footer** pointing users to source files and run instructions

2. Backend experience

The backend exposes one main endpoint:

- 'GET /api/products'

This endpoint returns:

- a 'banner' string for the hero highlights
- a 'curated' product array

Each product includes:

- 'id'
- 'name'
- 'price'
- 'currency'
- 'badge'
- 'category'
- 'description'
- 'delivery'
- 'priceFormatted'

The backend calculates 'priceFormatted' using 'Intl.NumberFormat("en-IN")', which keeps INR presentation consistent for the frontend.

ARCHITECTURE OVERVIEW

Frontend flow

1. 'frontend/src/main.jsx' mounts the React app into '#root'.
2. 'frontend/src/App.jsx' manages page state.
3. On first render, 'useEffect' calls 'fetch("http://localhost:5174/api/products")'.
4. The frontend stores:
 - 'products'
 - 'banner'
 - 'status'
5. 'useMemo' derives unique categories from the fetched product list.
6. The UI updates based on loading, ready, or error state.

Backend flow

1. 'backend/index.js' creates an Express app.
2. 'cors()' and 'express.json()' middleware are enabled.
3. A hard-coded in-memory 'products' array acts as the catalog.
4. '/api/products' formats prices and returns JSON.
5. The server listens on port '5174' by default.

DATA FLOW

The most important flow in the project is:

Express product data

- > /api/products response
- > React fetch in App.jsx
- > setBanner / setProducts / setStatus
- > category extraction with useMemo
- > product cards and category panels render on screen

This makes the project a good example of separating:

- ****data provisioning**** on the backend,
- ****state handling**** in React,
- and ****presentation styling**** in CSS.

KEY FILES AND RESPONSIBILITIES

'frontend/src/main.jsx'

- React entry point
- wraps the app in 'StrictMode'
- mounts '<App />'

'frontend/src/App.jsx'

- contains the page layout
- holds data-fetching logic
- derives category labels
- renders product cards
- handles loading and error UI

'frontend/src/App.css'

- provides the complete page styling
- defines layout, spacing, gradients, cards, chips, footer, and responsive rules

'frontend/src/index.css'

- defines base CSS variables and font imports

- sets base document styling

`'backend/index.js'`

- creates the API
- stores sample products
- formats INR prices
- returns the catalog payload

UI AND UX CHARACTERISTICS

The current interface is visually strong for a learning project:

- warm gradient hero section
- card-based merchandising layout
- responsive product grid
- clear visual grouping between content sections
- visible runtime states such as loading and backend connection status

The design communicates an ecommerce showcase rather than a complete shopping platform. That is appropriate for a training repo, but it also shows where future improvements can be made.

CURRENT STRENGTHS

- Simple and easy-to-follow architecture
- Clear separation between frontend and backend
- Useful example of React state + API integration
- Server-side price formatting reduces duplication in the client
- Responsive CSS structure already exists
- Product cards are reusable and data-driven

CURRENT GAPS AND LIMITATIONS

While the project is clean and understandable, it is still a demo implementation. Important limitations include:

- Only one API endpoint exists
- Product data is hard-coded in memory
- No database or file-based persistence
- No cart, filters, authentication, or checkout flow
- Minimal error handling on the backend
- No tests for API response shape
- No request logging
- No loading skeletons or empty-state polish
- Some text in the UI appears to have encoding issues such as '??' and '???'

These are useful talking points in an improvement document because they distinguish the current demo from a more production-ready store.

IMPROVEMENT IDEAS FROM 'TOP-UP-IDEAS'

The repository already includes a strong improvement roadmap. Below is a simplified explanation of each idea and why it matters.

1. Mega menu for Indian festivals

Add a richer navigation experience with hoverable dropdowns for festive and artisan collections.

****Why it helps:****

- improves discoverability
- makes the header feel more ecommerce-like
- introduces accessibility considerations such as ARIA states

2. Filterable product drawer

Add frontend state for:

- price range
- delivery speed
- category

****Why it helps:****

- turns the page from static showcase into an interactive catalog
- teaches derived state and conditional rendering
- improves product browsing usability

3. Localized offer banner

Replace the static status card or banner content with rotating offers such as:

- Diwali Flash Sale
- Metro Express Delivery
- Local Artisan Week

****Why it helps:****

- makes the hero area more dynamic
- is a good use case for timed state updates and CSS animations

4. Mini cart experience

Add cart state, totals, and 'localStorage' persistence.

****Why it helps:****

- introduces a real shopping interaction
- demonstrates state synchronization and side effects
- makes the project closer to an actual store flow

5. Backend polish

Move the catalog to a JSON file or mock DB, add logging, and validate responses with Jest or a script.

****Why it helps:****

- improves maintainability
- prepares the backend for growth
- introduces operational engineering habits

6. Technical documentation update

Run tools like ESLint or Lighthouse and capture findings in a report.

****Why it helps:****

- creates an evidence-based improvement process
- helps learners connect code quality with documentation

RECOMMENDED PRIORITY ORDER

If the team wants to improve the project in a practical sequence, this order makes sense:

1. Fix text encoding issues and improve copy consistency
2. Add backend logging and move product data to a JSON source
3. Add filterable product drawer
4. Add mini cart with local storage
5. Add rotating localized offer banner
6. Add mega menu and accessibility states
7. Run lint/performance review and update technical docs

This order starts with stability and clarity, then moves into richer UX features.

LOCAL RUN INSTRUCTIONS

Backend

```
cd module4/myntra-style-store/backend
npm install
npm start
```

The backend should run on:

`http://localhost:5174`

Frontend

```
cd module4/myntra-style-store/frontend  
npm install  
npm run dev
```

The frontend should run on:

<http://localhost:5173>

SUGGESTED TALKING POINTS FOR PRESENTATION OR HANDOVER

If this document is used in a class, demo, or internal handover, these are the best points to highlight:

- The website is a demo ecommerce storefront built for learning, not a full shopping platform.
- React handles UI composition and API-driven rendering.
- Express acts as a lightweight catalog service.
- The current codebase is intentionally small, so learners can understand the full request-response cycle quickly.
- The 'top-up-ideas' folder gives a ready-made roadmap for turning the demo into a more realistic store experience.

CONCLUSION

The Myntra-style store is a strong teaching project because it is easy to run, easy to read, and already structured around meaningful frontend-backend separation. The next phase should focus on improving realism: better data handling, richer interactions, clearer technical reporting, and more polished UX. With those improvements, the project can evolve from a good demo into a more complete portfolio-grade ecommerce example.